

EkuboProtocol Audit Scan

Apex Security Review

July 24, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
4	Audit Scan #1	4
4.1	Low Risk	4
4.1.1	EKUB3-60 - Governor executes tied proposals without a majority	4
4.1.2	EKUB3-2 - StakerV2 truncates short-period user stake shares before applying stake weight	5
4.1.3	EKUB3-19 - Token0 exact-input swaps materially underpay output at high prices	7
4.1.4	EKUB3-14 - Intermediate Q128 truncation strands material token0 deposits at the lower price boundary	9
4.1.5	EKUB3-24 - SDK exact-in routes default to no minimum output	11
4.2	Informational Risk	13
4.2.1	EKUB3-64 - SignedExclusiveSwap consumes unrestricted nonces on zero-amount no-op swaps	13
4.2.2	EKUB3-65 - Full uint32-epoch inactivity leaves Ve33 emission schedules active forever	15
4.2.3	EKUB3-1 - Airdrop burns valid claims when ERC20 transfer returns false	17
4.2.4	EKUB3-48 - Child proxies execute later authorized messages before earlier nonce-ordered messages	18
4.2.5	EKUB3-10 - SDK numeric signed-swap nonces round above 2^{53} and collide on-chain	20
4.2.6	EKUB3-46 - Ve33 zero-movement exact-input swaps bypass voter-fee accounting	22

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

This workspace aggregate report summarizes the client-visible findings across active scans for EkuboProtocol Audit Scan for starknet-contracts. Effective severities, statuses, notes, and comments are sourced from persisted client feedback at export time.

From Jul 20, 2026 to Jul 20, 2026 Cantina conducted a security review of [starknet-contracts](#) on commit hash [4b56b8188a5a72bfd9c38030e83010560f2e9cbd](#). The team identified a total of **11** issues:

Issues Found			
Severity	Count	Fixed	Acknowledged
Extreme Risk	0	0	0
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	5	2	0
Informational	6	1	5
Total	11	3	5

The following findings were acknowledged at the time of the review:

Acknowledged

- EKUB3-64 — SignedExclusiveSwap consumes unrestricted nonces on zero-amount no-op swaps
- EKUB3-65 — Full uint32-epoch inactivity leaves Ve33 emission schedules active forever
- EKUB3-1 — Airdrop burns valid claims when ERC20 transfer returns false
- EKUB3-48 — Child proxies execute later authorized messages before earlier nonce-ordered messages
- EKUB3-46 — Ve33 zero-movement exact-input swaps bypass voter-fee accounting

Findings without a Fixed or Acknowledged client label remain Open and are listed in their respective finding sections.

Cantina reviewed [starknet-contracts](#) holistically on commit hash [4b56b8188a5a72bfd9c38030e83010560f2e9cbd](#). At the time of this export, the included findings include open items alongside fixed or acknowledged statuses and no new vulnerabilities were identified.

2.1 Scope

This workspace aggregate report covers the following active scan snapshots:

- Audit Scan #1
 - <https://github.com/EkuboProtocol/starknet-contracts> - branch main - commit [4b56b8188a5a](#)
 - <https://github.com/EkuboProtocol/governance> - branch main - commit [fc9b00535928](#)
 - <https://github.com/EkuboProtocol/evm-contracts> - branch main - commit [7b8b0a070e2f](#)
 - <https://github.com/EkuboProtocol/yul-router> - branch main - commit [e1c806a3c80d](#)

EkuboProtocol Audit Scan for starknet-contracts was reviewed by Cantina across 1 scan. This workspace aggregate export covers 1 active scan.

This export includes 11 client-visible findings, grouped by effective severity and using persisted client statuses for report presentation.

3 Findings

4 Audit Scan #1

Created: July 20, 2026

Repositories

- <https://github.com/EkuboProtocol/starknet-contracts> - branch main - commit 4b56b8188a5a
- <https://github.com/EkuboProtocol/governance> - branch main - commit fc9b00535928
- <https://github.com/EkuboProtocol/evm-contracts> - branch main - commit 7b8b0a070e2f
- <https://github.com/EkuboProtocol/yul-router> - branch main - commit e1c806a3c80d

4.1 Low Risk

4.1.1 EKUB3-60 - Governor executes tied proposals without a majority

✓ Low

Status: Fixed

Original severity: Low

Executive Summary

Governor.execute treats a tie between yea and nay voting weight as a passing majority. Any proposal with `yea == nay` and `yea >= quorum` passes the execution predicate, even though the proposal has no majority support. Because the proposal carries an arbitrary, hash-committed call array, this permits execution of the calls from a quorum-level tied vote that the documented governance rule rejects. The defect is a boundary-condition authorization failure in the final state transition: the call hash and timing checks remain correct, but the approval predicate is too permissive.

Details

`governance/src/governor.cairo:388-390` performs the final vote checks:

```
assert(proposal.yea >= config.quorum, 'QUORUM_NOT_MET');
assert(proposal.yea >= proposal.nay, 'NO_MAJORITY');
```

The second comparison must be strict for a majority rule. The execution path is:

- A proposer creates a proposal after satisfying the configured creation threshold. The proposal commits the exact call array through `calls_hash`.
- During the voting period, voters cast yes and no votes. Their historical weights are added to `proposal.yea` and `proposal.nay`.
- After the delay, an executor supplies the same calls. `execute` verifies the hash, timing, and quorum, then accepts `yea == nay` because of `>=`.
- The Governor marks the proposal executed and dispatches every committed call, including self-upgrade, L1-message, or owner-proxy calls.

For example, with `quorum = 500`, `yea = 500` and `nay = 500` passes both assertions. The repository test at `governance/src/governor_test.cairo:687-727` checks only `nay = quorum + 1` and therefore does not exercise the equality boundary.

Impact Cascade

- A proposal with no majority support can enter the executed state.
- A tied proposal can dispatch arbitrary target and calldata already committed by that proposal.
- A tied self-call can change Governor configuration or class hash, and a tied L1-message call can reach the downstream owner proxy.

Assumptions and Uncertainties

- The deployed configuration uses the documented majority rule represented by the `NO_MAJORITY` check.
- Voting weight can be split between yea and nay such that both equal or exceed quorum, which is possible with multiple delegated voters.
- The call array itself is otherwise the intended proposal payload; this issue does not rely on a hash collision or malformed calldata.

How will the bug recipient respond?

"A proposal still needs quorum-level yea votes, so the yes side has enough authority."

Quorum and majority are separate approval conditions in the contract and documentation. A quorum-level yes vote is explicitly insufficient when an equal no vote exists; accepting equality removes the majority condition rather than merely checking quorum.

Why did tests miss this issue?

The negative execution test supplies more nay weight than yea weight. It verifies only the strict-failure side of the comparison and has no case where `yea == nay` at quorum.

Recommendation

Use a strict comparison for the majority predicate:

```
assert(proposal.yea > proposal.nay, 'NO_MAJORITY');
```

Add a regression case with `yea == nay == quorum` and assert that execution reverts without changing execution state or dispatching calls.

References

- [governance/src/governor.cairo#L377-L405](#)
- [governance/src/governor_test.cairo#L687-L727](#)
- [governance/README.md#L36-L48](#)

Client Feedback

Ties are very unlikely with tokens that have 18 decimals

4.1.2 EKUB3-2 - StakerV2 truncates short-period user stake shares before applying stake weight



Low

Status: Fixed by [PR #77](#)

Original severity: Low

Executive Summary

`StakerV2.get_user_share_of_total_staked_over_period` calculates a user's time-weighted share by first converting the Q128 reciprocal-total-stake accumulator into an integer percentage and only then multiplying by the user's stake. The truncation happens before the stake is applied. Consequently, a user who owns the entire active stake can receive a zero result for a short interval even though the mathematically correct share is 100%. This is not bounded one-unit dust: for normal token-sized totals and short accounting windows, the entire share is discarded. The function is a public V2 accounting surface reachable after ordinary `stake_amount` and `withdraw_amount` mutations, so any governance, reward, or distribution consumer using this share getter can under-credit users and misattribute the missing value.

Details

The susceptible expression is at `governance/src/staker_v2.cairo#L465-L473`:

```
fn get_user_share_of_total_staked_over_period(
    self: @ContractState, staked: u128, start: u64, end: u64,
) -> u128 {
    assert(end > start, 'ORDER');

    let start_snapshot = self.get_seconds_per_total_staked_sum_at(start);
```

```

    let end_snapshot = self.get_seconds_per_total_staked_sum_at(end);

    staked * ((end_snapshot - start_snapshot) * 100).high / (end - start).into()
}

```

`get_seconds_per_total_staked_sum_at` stores `seconds / total_staked` in Q128 using u256 { low: 0, high: `seconds_diff` } / `total_staked`. The `.high` operation in the return expression therefore computes `floor(seconds * 100 / total_staked)` before `staked` is multiplied in. The correct order is to multiply the Q128 accumulator by `staked * 100` before taking the high word, or to retain sufficient precision through the final division.

A concrete reachable state is one user staking 1,000 units at timestamp 10 and querying the interval [10, 11]: the true share is $1,000 / 1,000 = 100\%$. The accumulator difference is approximately $2^{128} / 1000$; multiplying by 100 still has a zero high word, so the function returns zero. The same state can be created through the public `stake_amount` path at `governance/src/staker_v2.cairo#L264-L285` and queried without any privileged action.

Impact Cascade

- A fully participating user can receive a zero share for short periods.
- Reward, fee, or governance distributions based on this getter can under-credit users by the entire amount attributable to the interval.
- The unassigned amount can remain in the distributor or be allocated inconsistently to other participants, violating time-weighted stake attribution.

Validation

- Is the input attacker-controlled? Yes. `start`, `end`, and the public view call are caller supplied; the stake and total are established through permissionless staking mutations.
- Does the path reach production code? Yes. The expression is in the public `IstakerV2` ABI and the underlying stake log is written by `stake_amount` and `withdraw_amount`.
- Is the discrepancy material rather than dust? Yes. With total and user stake of 1,000 over one second, the result is 0 instead of 100.
- Does the calculation depend on nonstandard tokens or reentrancy? No. Standard token transfers and ordinary staking are sufficient.
- Is the issue covered by the existing tests? No. The V2 share-related tests exercise totals such as 10 and 20 over multi-second periods; they do not preserve precision by testing totals above `100 * period`.

Valid Argument

The function's stated purpose is to calculate a user's share of total staked value over a period. Its fixed-point accumulator is intentionally designed to preserve the fractional reciprocal, but the implementation discards that precision before applying the user's stake. The error is directly reachable and can eliminate all attribution for a valid, nonzero share.

Invalid Counter-Argument

The current Governor implementation calls the older `Istaker` delegated-average methods and does not call this V2-specific share getter. A triager may therefore argue that no current in-repository execution path consumes the incorrect return value. That limits the demonstrated downstream impact to V2 integrations or deployments that use this public accounting API, but it does not make the getter's result correct or the reachable accounting error bounded.

Why did tests miss this issue?

Existing V2 tests use small totals and intervals long enough that `seconds * 100 / total` is at least one before the high word is taken. They do not test the mathematically important case where the user's stake multiplication is required to preserve the fractional result.

Recommendation

Preserve the Q128 precision until after multiplying by the user's stake. For example, compute the Q128 numerator as `(end_snapshot - start_snapshot) * staked * 100`, take its high word, and divide by the period, with an explicit wide-intermediate overflow policy. Alternatively, multiply the reciprocal percentage by `staked` before converting it to an integer percentage.

References

- governance/src/staker_v2.cairo#L264-L285
- governance/src/staker_v2.cairo#L371-L400
- governance/src/staker_v2.cairo#L465-L473
- governance/src/staker_v2_test.cairo#L543-L587

Client Feedback

We removed this function altogether since it was not used

4.1.3 EKUB3-19 - Token0 exact-input swaps materially underpay output at high prices



Status: Open

Original severity: Low

Executive Summary

The token0 exact-input branch computes an intermediate quantity ($\text{liquidity} \ll 128$) / sqrt_ratio with integer floor before adding the input amount. It then divides the numerator by that truncated denominator. At supported near-maximum prices, this intermediate quantity can be zero for valid nonzero liquidity. The resulting next square-root price is materially too high, so the calculated token1 output is materially too small. Core nevertheless accounts the full token0 input and Router settles the reduced token1 output as a successful swap. This transfers the missing output to the pool/LPs and makes quotes and user value attribution incorrect. The issue is distinct from the token1 Q128 truncation: it is in `next_sqrt_ratio_from_amount0`, where the denominator is truncated before the input is added.

Details

The vulnerable calculation is:

```
pub fn next_sqrt_ratio_from_amount0(
    sqrt_ratio: u256, liquidity: u128, amount: i129,
) -> Option<u256> {
    ...
    let numerator1 = u256 { high: liquidity, low: 0 };
    ...
    let (denominator_p1, _) = DivRem::div_rem(
        numerator1, sqrt_ratio.try_into().unwrap(),
    );
    let denominator = denominator_p1 + amount.mag;
    let (quotient, remainder) = DivRem::div_rem(
        numerator1, denominator.try_into().unwrap(),
    );
    ...
}
```

For token0 exact-input swaps, `swap_result` passes `price_impact_amount` to this function and uses the returned ratio to calculate token1 output:

```
let sqrt_ratio_next_from_amount = if (is_token1) {
    next_sqrt_ratio_from_amount1(sqrt_ratio, liquidity, price_impact_amount)
} else {
    next_sqrt_ratio_from_amount0(sqrt_ratio, liquidity, price_impact_amount)
};
...
let calculated_amount_excluding_fee = if (is_token1) {
    amount0_delta(sqrt_ratio_next, sqrt_ratio, liquidity, amount.sign)
} else {
    amount1_delta(sqrt_ratio_next, sqrt_ratio, liquidity, amount.sign)
};
```

A reachable boundary example uses a pool initialized at `max_tick() - 1`, a full-range position with liquidity 2^{63} , a zero-fee pool, and an exact-input amount0 of 1. The supported maximum square-root ratio is greater than 2^{191} , while $\text{liquidity} * 2^{128}$ is exactly 2^{191} , so the source computes:

```
floor((liquidity * 2**128) / sqrt_ratio) = 0
source next ratio = ceil((liquidity * 2**128) / 1) = 2**191
```

The full-precision denominator is approximately $0.5 + 1$, so the correct next ratio is approximately $2^{190.415}$, not 2^{191} . At the maximum supported ratio, the source output is approximately $8.507e37$ token1 units while the full-precision output is approximately $1.134e38$, a deficit of approximately $2.836e37$ units (about 25%). Moving from `max_tick()` to `max_tick()-1` keeps the same order of magnitude and makes the full-range liquidity active.

The public Router reaches this branch through `Core.swap` and settles the returned deltas:

```
let delta = core.swap(
  node.pool_key,
  SwapParameters {
    amount: token_amount.amount,
    is_token1: is_token1,
    sqrt_ratio_limit: sqrt_ratio_limit,
    skip_ahead: node.skip_ahead,
  },
);
...
handle_delta(core, token_amount.token, -token_amount.amount, recipient);
handle_delta(core, first.token, first.amount, recipient);
```

Core's swap loop persists the reduced calculated amount in the token1 delta and the Router pays/withdraws exactly that result. There is no invariant check against the full-precision amount or a bounded-rounding guarantee.

Five Validation Probes

- Does production code use the token0 helper? Valid: `swap_result` calls `next_sqrt_ratio_from_amount0` whenever `is_token1` is false, and Core uses `swap_result` for every locked swap. Invalid: this would be helper-only if no production caller reached it; the direct branch disproves that.
- Can the intermediate floor be zero with supported values? Valid: `max_tick()-1` has a square-root ratio above 2^{191} , while liquidity 2^{63} makes the numerator exactly 2^{191} . Invalid: the state would be unreachable if the tick or liquidity were outside bounds; `max_tick()-1`, u128 liquidity, and full-range bounds are supported.
- Is the discrepancy material rather than one-unit dust? Valid: with amount0 1, the source ratio is 2^{191} and the full-precision ratio is about $2^{190.415}$, producing roughly a 25% token1-output deficit. Invalid: ordinary fixed-point rounding could explain only a bounded raw-unit difference, not this ratio-scale gap.
- Does settlement preserve the discrepancy? Valid: Core constructs the token1 delta from `amount1_delta` at the truncated ratio and Router settles that delta, so the unreturned token1 remains in Core custody. Invalid: a later settlement correction would mitigate it, but no such correction exists in Core or Router.
- Is there an unprivileged production trigger with economic impact? Valid: any public Router swapper can select a supported pool and exact-input token0 path; the user receives materially less output and LP accounting receives the excess. Invalid: no privilege, malicious token, reentrancy, or unsupported token behavior is required, although the extreme price/liquidity configuration is uncommon.

Impact Cascade

- Exact-input token0 traders receive materially less token1 than the pool invariant implies.
- The retained token1 is treated as pool/LP value even though it is caused by arithmetic truncation.
- Quotes and execution accounting diverge at supported high-price, low-intermediate-denominator states.
- Repeated or large swaps can create material user losses rather than one-unit rounding dust.

Assumptions and Uncertainties

- The pool permits the supported `max_tick()-1` price and a nonzero full-range liquidity position.
- The token pair follows standard ERC20 behavior.
- The numeric comparison uses the full-precision AMM formula with conservative output flooring; exact source ratios at `max_tick()-1` change only the final digits, not the material gap.
- The effect favors LP custody over the swapper; it is reported as a protocol accounting/value-attribution failure rather than an attacker-controlled drain.

How will the bug recipient respond?

"This is just fixed-point rounding at an extreme price."

The intermediate floor is not a bounded final-output rounding step. At valid inputs it changes the denominator from approximately 1.5 to 1, producing an approximately 25% output difference. The existing tests exercise ordinary center-price values and do not cover the supported near-maximum ratio where the intermediate quotient is below one.

Why did tests miss this issue?

The ratio tests cover `sqrt_ratio = 2**128` with liquidity around `10**6` and ordinary amounts. The swap tests likewise use center-price pools. They do not combine a near-maximum supported square-root ratio with liquidity small enough that `(liquidity << 128) / sqrt_ratio` floors to zero.

Recommendation

Use a full-precision rational calculation for the denominator instead of flooring before adding the input. Preserve the numerator and denominator through the final division, then round the final next ratio conservatively toward the current price. For example, compute the equivalent of `ceil((liquidity * sqrt_ratio * 2**128) / (liquidity * 2**128 + amount * sqrt_ratio))` with a checked 512-bit muldiv, rather than dividing `liquidity << 128` by `sqrt_ratio` first.

References

- starknet-contracts/src/math/sqrt_ratio.cairo#L9-L52
- starknet-contracts/src/math/swap.cairo#L65-L152
- starknet-contracts/src/core.cairo#L914-L999
- starknet-contracts/src/router.cairo#L81-L131
- starknet-contracts/src/tests/math/sqrt_ratio_test.cairo#L6-L50

Client Feedback

These prices are not practical and in most cases the rounding only loses negligible amounts

4.1.4 EKUB3-14 - Intermediate Q128 truncation strands material token0 deposits at the lower price boundary

✓ Low

Status: Open

Original severity: Low

Executive Summary

`max_liquidity_for_token0()` truncates the Q128 product `sqrt_ratio_lower * sqrt_ratio_upper / 2^128` before using it to invert the token0 delta formula. That is conservative for ordinary prices, but it is not a one-unit rounding effect at the protocol's valid lower price boundary. For the valid 128-tick interval from the maximum usable lower tick (-88368108) to -88367980, the exact factor is approximately 1.42595 while the intermediate integer is 1. The resulting liquidity is therefore only about 70.13% of the liquidity supported by the supplied token0.

`Positions.deposit()` computes liquidity from the helper's complete token balance, writes only that lower liquidity to Core, and pays only the resulting delta. The remainder stays in the Positions contract. The normal `mint_and_deposit()` path does not clear it, while the public clear path transfers helper balances to the caller. A valid boundary-range deposit can therefore strand roughly 29.87% of a material token0 deposit, and a later unprivileged caller can capture that residual under the helper's balance-attribution design.

Details

The intended inverse is:

```
L = amount0 * sqrt_lower * sqrt_upper /
    (2^128 * (sqrt_upper - sqrt_lower))
```

The implementation instead performs two floors:

```

let numerator_1 = muldiv(sqrt_ratio_lower, sqrt_ratio_upper,
    u256 { high: 1, low: 0 }, false);
let result = muldiv(amount.into(), numerator_1,
    (sqrt_ratio_upper - sqrt_ratio_lower), false);

```

At lower usable tick -88368108 and upper tick -88367980 (128 ticks apart), $\text{sqrt_lower} * \text{sqrt_upper} / 2^{128} \approx 1.42595149$, so line 15 stores `numerator_1 = 1`. The token0-limited branch is selected whenever the current price is at or below the lower bound. Consequently, for a large token0 amount the minted liquidity is based on $\text{amount0} / (\text{sqrt_upper} - \text{sqrt_lower})$ instead of $1.42595149 * \text{amount0} / (\text{sqrt_upper} - \text{sqrt_lower})$. `liquidity_delta_to_amount_delta()` then requires only approximately $\text{amount0} / 1.42595149$, leaving approximately 29.87% of the input balance unconsumed. For example, a 1,000,000,000,000,000-unit token0 balance leaves roughly 298,700,000,000,000 units, far beyond dust.

The production path is:

- `mint_and_deposit()` mints the NFT and calls `deposit()`.
- `deposit()` reads the full Positions token0/token1 balances.
- The Core locker computes `max_liquidity()` and enters its `sqrt_ratio <= lower` token0 branch.
- Core persists the under-sized position, then `core.pay()` transfers only the computed delta.
- The residual remains in the helper; `mint_and_deposit_and_clear_both()` is an optional separate path, while generic `clear()` can be called by a later caller.

Five validation questions

- **Is the arithmetic boundary reachable?** Yes. The cited ticks are within the documented maximum usable bounds, the 128-tick interval is valid for a pool with tick spacing 1, and a pool price at the minimum boundary is valid.
- **Does the truncation alter an asset-bearing result?** Yes. It reduces supported liquidity by about 29.87%, so the Core payment delta is smaller by a material amount rather than by one raw unit.
- **Does the result reach persistent state?** Yes. Positions writes the returned liquidity through `Core.update_position()` before paying the computed delta.
- **Who owns the unconsumed balance?** No operation-specific owner is recorded; it remains on the Positions contract and is transferable through its public clearing path.
- **Can a caller avoid the effect?** A caller can use the optional clear-both helper, but the ordinary production `mint_and_deposit()` path leaves the residual behind. This is a mitigation convention, not a correction to the inverse formula.

Valid and Invalid Arguments

Valid: The error is induced by supported `u128` amounts and valid tick bounds, persists after the transaction, and is material at ordinary token decimal scales. The result is not merely a quote discrepancy: the contract consumes and accounts for less of the balance than the public deposit operation uses as its input.

Invalid: The issue does not apply to normal mid-range prices where the intermediate Q128 product is large, nor to a caller who immediately invokes the optional clear-both helper. It also does not rely on fee-on-transfer or malicious-token behavior. Those limitations narrow the affected boundary but do not remove the valid production path.

How will the bug recipient respond?

> "Users can call the clear-both helper themselves, so no funds are lost."

That response does not fix the math or the ordinary `mint_and_deposit()` lifecycle: the helper has no binding between the residual and the funding operation, and a later caller can clear it before the depositor does. Even when the depositor reclaims the residual, the requested deposit operation deterministically under-mints at a supported price boundary.

Why did tests miss this issue?

Existing max-liquidity tests cover full-range, half-range, and a concentrated example, but do not exercise the maximum usable lower tick with a narrow interval where the intermediate Q128 product is close to 1. They also assert conservative liquidity in terms of the resulting delta, not that a deposit consumes the intended amount at both fixed-point extremes.

Recommendation

Compute the full-width numerator and divide only once, or use a muldiv sequence that preserves the fractional Q128 factor. For example, calculate `amount0 * sqrt_lower * sqrt_upper / (2^128 * (sqrt_upper - sqrt_lower))` with the existing 512-bit muldiv helper and floor only the final result. Then either refund any intentional balance remainder to the authenticated deposit caller or bind helper balances to the deposit operation so residuals cannot be claimed by another caller.

References

- `src/math/max_liquidity.cairo#L8-L23`
- `src/math/max_liquidity.cairo#L43-L60`
- `src/positions.cairo#L283-L319`
- `src/positions.cairo#L792-L810`
- `src/positions.cairo#L943-L967`

Client Feedback

The leftovers are always cleared and this price range is extreme so this rarely happens

4.1.5 EKUB3-24 - SDK exact-in routes default to no minimum output

✓ **Low**

Status: Open

Original severity: Low

Executive Summary

`encodeRoutes()` emits an exact-in route with a zero calculated-amount threshold whenever the caller omits `calculatedAmountThreshold`. The Yul runtime treats zero as “no slippage check”. This is exploitable on a valid, permissionlessly creatable Core pool whose fee is `type(uint64).max`: for any positive input below 2^{64} , Core’s rounded-up fee equals the entire input, the swap returns zero calculated output, and the Yul route still settles the input successfully. A caller using the SDK’s normal convenience path can therefore pay the full input while receiving nothing unless every integration supplies its own threshold.

Details

The SDK selects zero for exact-in routes:

```
const threshold =
  calculatedAmountThreshold ??
  (isExactOut === true ? MIN_CALCULATED_AMOUNT_THRESHOLD : 0n);
```

It encodes that zero into the packed route header. In `locked()`, the runtime only executes the slippage comparison after a truthiness check on `threshold`, so a zero threshold bypasses both the sign check and `slt(totalCalculated, threshold)` failure path:

```
if threshold {
  if exactOutKnown { ... }
}
if slt(totalCalculated, threshold) { revert(...) }
```

Core accepts the fee field without a fee upper-bound validation. For a seeded ordinary pool with `fee = 0xffffffffffffffff` and a positive input such as 1 ether, `computeFee()` rounds the fee to 1 ether; `priceImpactAmount` becomes zero and the no-price-movement branch consumes the whole input as fees while leaving `calculatedAmount` at zero. Core returns (`deltaIn = 1 ether`, `deltaOut = 0`). Yul’s `nextFromUpdate()` accepts the input leg, aggregates `totalCalculated = 0`, skips the zero threshold, calls `pay()` for the full specified input, and withdraws zero output. The lock debt therefore closes and the transaction succeeds.

The exact bytes are deterministic for a one-hop route: the SDK header contains ... `0000000000000000000000000000000000000000000000000000000000000000` at the 16-byte threshold field, followed by the ordinary type-0 hop. No malformed calldata or non-standard token behavior is required.

Impact Cascade

- SDK-generated exact-in transactions can settle with zero output while spending the full caller input.
- A permissionless pool creator can publish a standard Core pool with a near-100% fee and make the failure deterministic for integrators that omit the threshold.
- Aggregators or UIs that rely on the SDK convenience default lose the caller's input to the pool fee bucket instead of failing before settlement.

Assumptions and Uncertainties

- An integration uses `encodeRoute()/encodeRoutes()` without passing a minimum output threshold.
- The route selects an initialized ordinary Core pool whose configured fee is near 100%; Core does not validate that fee field.
- The caller has approved and funded the specified token, as required for any exact-in route.
- The finding concerns the missing default safety invariant; a caller that explicitly supplies a threshold of at least one output unit is protected by the existing Yul check.

How will the bug recipient respond?

> "A zero minimum output is an explicit user choice, and users can provide a threshold."

That response is valid for callers who consciously choose zero. It does not address the SDK's documented convenience constructor silently selecting the unsafe value for every omitted exact-in threshold, nor the deterministic zero-output path on a valid high-fee pool. The safe behavior is to require an explicit threshold or use a fail-closed default.

Why did tests miss this issue?

SDK tests only assert that ordinary positive-output routes encode successfully. The integration tests use a moderate fee and assert `calculatedAmount > 0`; they do not exercise an omitted exact-in threshold against the maximum valid Core fee or assert that successful routes produce nonzero output.

Recommendation

Require `calculatedAmountThreshold` for exact-in routes, or make the API accept an explicit opt-in for an unbounded minimum. A fail-closed alternative is to reject an omitted threshold rather than encoding zero:

```
if (!isExactOut && calculatedAmountThreshold === undefined) {  
  throw new Error("calculatedAmountThreshold is required for exact-in routes");  
}
```

References

- [yul-router/sdk/src/index.ts#L221-L246](#)
- [yul-router/src/YulRouter.yul#L254-L269](#)
- [yul-router/lib/evm-contracts/src/Core.sol#L638-L738](#)
- [yul-router/lib/evm-contracts/src/types/poolConfig.sol#L257-L272](#)

4.2 Informational Risk

4.2.1 EKUB3-64 - SignedExclusiveSwap consumes unrestricted nonces on zero-amount no-op swaps



Status: Acknowledged

Original severity: Informational

Executive Summary

SignedExclusiveSwap.handleForwardData consumes a signed quote nonce after calling Core.swap, but neither the Yul route parser nor the extension rejects a zero swap amount. Core.swap intentionally treats amount == 0 as a successful no-op and returns a zero PoolBalanceUpdate. For any valid signature whose minBalanceUpdate components are non-positive (including a permissive zero lower bound) and whose locker authorization is unrestricted, an unprivileged caller can submit a zero-amount signed hop. Signature validation succeeds, the zero update satisfies the signed lower bound, and _consumeNonce permanently marks the one-time nonce as used. The caller spends no asset and receives no asset, but the controller's quote is unavailable to the intended fill until a new signature is issued.

Details

In the Yul path, a route amount is decoded as a signed int128 and zero is accepted:

```
let currentAmount := sar(128, calldataload(offset))
...
let update := signedExclusiveSwap(..., currentAmount, ...)
```

The SDK likewise accepts specifiedAmount == 0 and emits the signed hop without adding any nonzero-amount constraint. The hop forwards SwapParameters.amount == 0 to the extension.

```
(PoolBalanceUpdate balanceUpdate, PoolState stateAfter) = CORE.swap(0, poolKey, params);

if (
    balanceUpdate.delta0() < minBalanceUpdate.delta0() || balanceUpdate.delta1() < minBalanceUpdate.
    delta1()
) {
    revert MinBalanceUpdateNotMet(minBalanceUpdate, balanceUpdate);
}

_consumeNonce(meta.nonce());
```

Core.swap validates the pool and hooks, then skips its movement loop when the amount is zero:

```
// 0 swap amount or sqrt ratio limit == sqrt ratio is no-op
if (amountRemaining != 0 && stateAfter.sqrtRatio() != sqrtRatioLimit) {
    // swap movement and debt updates
}
```

It therefore returns (0, 0). A signed payload with minBalanceUpdate == bytes32(0) (or any component-wise non-positive bound) accepts this update. With the documented zero locker selector, meta.isAuthorized(original) also accepts any caller. The extension then consumes every non-sentinel nonce despite no balance movement. The route settles totals of zero, so it does not need an allowance or token balance.

A concrete route is a one-hop Yul route with specifiedAmount = 0, specifiedToken = poolKey.token0, calculatedToken = poolKey.token1, threshold 0, hop type 4, the genuine pool key, unrestricted signed meta, the controller signature over (poolId, meta, bytes32(0)), and a non-sentinel nonce. It reaches the production Core.forward -> SignedExclusiveSwap.handleForwardData -> Core.swap path and leaves the nonce bit set.

Impact Cascade

- A remote caller can invalidate a still-valid, one-time controller quote without paying the quoted asset or satisfying the intended route amount.
- The intended user or aggregator cannot execute that quote afterward because _validateNonceAvailable rejects the consumed nonce.

- Publicly broadcast or mempool-visible unrestricted quotes can be selectively grieved at negligible capital cost, forcing replacement signatures and disrupting exclusive-liquidity execution.

Assumptions and Uncertainties

- The controller issues at least one valid quote with an unrestricted locker and a lower bound that accepts a zero update; this is supported by the signed type and its component-wise lower-bound semantics.
- The quote has a non-sentinel nonce, as required for one-time execution.
- The attacker can observe the signature through the broadcast event, a submitted route, or the public transaction relay before intended execution.
- This does not rely on malformed tokens, privileged roles, reentrancy, or a compromised controller.

How will the bug recipient respond?

> “A zero lower bound intentionally permits flexible execution, and the caller is allowed to execute any valid signed quote.”

That does not explain consuming a one-time nonce when Core performed no swap and the route settled zero assets. The signed authorization is for an economic execution, while the no-op path provides no economic execution and can be submitted solely to destroy replay state.

Why did tests miss this issue?

The signed-hop tests use nonzero swap parameters and seeded liquidity, and the replay test checks a nonce after a successful nonzero fill. They do not exercise a zero route amount with a permissive signed minimum, nor assert that no-op paths leave the nonce available.

Recommendation

Reject no-op signed executions before consuming the nonce. The safest invariant is to require the returned balance update to contain movement:

```
if (balanceUpdate.delta0() == 0 && balanceUpdate.delta1() == 0) revert NoOpSwap();
_consumeNonce(meta.nonce());
```

The route/periphery may additionally reject zero specified amounts for signed hops. The same no-op check should cover a nonzero amount paired with a `sqrRatioLimit` equal to the current pool ratio.

References

- [evm-contracts/src/extensions/SignedExclusiveSwap.sol#L192-L239](#)
- [evm-contracts/src/Core.sol#L527-L545](#)
- [yul-router/src/YulRouter.yul#L93-L110](#)
- [yul-router/sdk/src/index.ts#L99-L126](#)
- [evm-contracts/src/types/signedSwapMeta.sol#L22](#)

Client Feedback

No reason to do this except to grieve a user with a valid nonce

4.2.2 EKUB3-65 - Full uint32-epoch inactivity leaves Ve33 emission schedules active forever

✓ **Informational**

Status: Acknowledged

Original severity: Informational

Executive Summary

Ve33 stores the last-accrued emission timestamp as only its low 32 bits. If a funded schedule is left untouched for one complete 2^{32} -second epoch, recovery advances to the next timestamp with the same low bits, which is after the schedule's real end. The accrual loop then searches only forward from that later timestamp, never consumes the old end event, and retains the schedule's positive rate. A later unprivileged user can create a normal stake, vote, and LP position, touch the pool, and claim rewards at the stale rate. The original schedule funded only its short interval; once that balance is exhausted, claims can debit the same saved-balance bucket that backs unrelated Ve33 stake principal.

Details

The vulnerable recovery is:

```
function realEmissionTimeAtOrBeforeNow(Ve33GlobalEmissionState state)
    view
    returns (uint256 realTime)
{
    uint32 time = state.lastAccrued();
    unchecked {
        realTime = block.timestamp - (uint32(block.timestamp) - time);
    }
}
```

A schedule created at T with end $E < T + 232$ leaves the packed last-accrued field equal to $\text{uint32}(T)$ if no emission-touching path runs. At $T + 232 + d$, the helper returns $T + 2^{32}$. Because E is behind this cursor, it is never found or cleared:

```
uint256 lastAccruedTime = globalEmissionState.realEmissionTimeAtOrBeforeNow();
while (time != block.timestamp) {
    (uint256 eventTime, bool initialized) =
        _searchForNextEmissionTime(lastAccruedTime, time, block.timestamp);
    if (initialized) {
        rate = uint160(uint256(int256(uint256(rate))
            + _emissionRateDeltaAtTime(eventTime)));
        _setEmissionRateDeltaAtTime(eventTime, 0);
        _flipEmissionTime(eventTime);
    }
    time = eventTime;
}
```

A later vote first touches accrual while total vote weight is zero, advancing the cursor without assigning growth; the next pool touch with a live voter realizes growth from the stale rate. `claimRewards` subtracts that amount from the shared stake-token saved balance, which also contains all staked principal. Root cause: a 32-bit timestamp is recovered without an ambiguity check or persisted epoch.

Impact Cascade

- An end-time bitmap entry becomes permanently unreachable after the epoch alias.
- The expired stream's rate remains active for subsequent accruals.
- Later LP claims are calculated for emissions never funded by the schedule.
- Excess claims can consume unrelated Ve33 stake principal and make later withdrawals or reward claims revert.

Assumptions and Uncertainties

- A funded schedule and its end event exist when the system becomes dormant.
- No state-changing Ve33 path that calls `accrueEmissions` executes for one full 2^{32} -second epoch.
- After the alias, an unprivileged actor can create a supported stake, vote, and Ve33 LP position, then use permissionless pool-touch and claim paths.
- The claim is sized or split below the aggregate saved balance; otherwise Core correctly reverts.

- This is a long-horizon precondition, but the resulting stale-rate accounting is deterministic and permanent once reached.

Five probing questions

- Does the helper identify the correct full timestamp? Valid: it uses only low 32 bits, so T and $T+2^{32}$ are indistinguishable. Invalid: selecting the current epoch does not prove the packed state was updated in that epoch.
- Can the old end event be processed after recovery? Valid: the search starts at the recovered timestamp and never scans backward. Invalid: the bitmap bit remains, but no production path searches older times.
- Does the stale rate persist? Valid: rate changes only in the initialized-event branch, and the skipped event is never cleared. Invalid: unrelated future events may alter rate but do not repair the old end transition.
- Can a public user reach a value-bearing sink? Valid: staking, voting, LP management, accumulation, and claims are public subject to ordinary token ownership. Invalid: oversized claims revert, so the attacker claims incrementally.
- Is the excess backed by the original schedule? Valid: the original saved balance covers only its scheduled amount, while stale-rate growth can consume the bucket's principal component. Invalid: left-over scheduled tokens may cover early claims.

How will the bug recipient respond?

> "A 136-year inactive period is outside the expected deployment lifetime."

That lowers likelihood but does not make the packed-state transition safe. Once reached, the contract silently converts an expired stream into a perpetual rate with no recovery path.

Why did tests miss this issue?

The Ve33 rollover test advances across a single 32-bit boundary while touching the pool at the scheduled end, so the end event is consumed before becoming unreachable. It does not leave a pending end event untouched for an entire epoch and then exercise a post-alias claim.

Recommendation

Store the full last-accrued timestamp, or persist an epoch counter with the low 32-bit value. If storage must remain packed, detect an ambiguous or over-epoch elapsed time and revert before advancing the cursor. Never let the search cursor move beyond unprocessed bitmap events.

References

- `evm-contracts/src/types/ve33GlobalEmissionState.sol#L22-L30`
- `evm-contracts/src/extensions/Ve33.sol#L917-L946`
- `evm-contracts/src/extensions/Ve33.sol#L1123-L1148`
- `evm-contracts/src/extensions/Ve33.sol#L765-L797`
- `evm-contracts/src/Ve33Positions.sol#L416-L425`

Client Feedback

136 years without touches will not happen

4.2.3 EKUB3-1 - Airdrop burns valid claims when ERC20 transfer returns false

✓ **Informational**

Status: Acknowledged

Original severity: Informational

Executive Summary

The Airdrop contract treats the call to `IERC20.transfer` as successful without checking its boolean return value. The single-claim path marks the claim bitmap before making the unchecked transfer, and the batch path marks every selected bit before iterating over unchecked transfers. ERC20-compatible tokens are allowed to return `false` to signal a failed transfer instead of reverting. In that case the call returns success, the claimant's entitlement is permanently marked as claimed, and no tokens are delivered. The issue affects the normal public Merkle-claim flow and is not dependent on fee-on-transfer behavior, reentrancy, or malformed proofs.

Details

The single-claim path at `governance/src/airdrop.cairo#L167-L185` does this:

```
self.claimed_bitmap.write(word, bitmap | exp2(index.try_into().unwrap()));
self.token.read().transfer(claim.claimee, claim.amount.into());

self.emit(Claimed { claim });
true
```

The batch path at `governance/src/airdrop.cairo#L210-L235` first persists `claimed_bitmap` and then calls `token.transfer` for every unclaimed entry, again ignoring the returned `bool`:

```
self.claimed_bitmap.write(word, bitmap);

while let Option::Some(claim) = unclaimed.pop_front() {
    token.transfer(claim.claimee, claim.amount.into());
    self.emit(Claimed { claim });
};
```

The interface explicitly models the ERC20 boolean return at `governance/src/interfaces/erc20.cairo#L7-L11`. Therefore, a token that returns `false` for an insufficient balance or another ordinary transfer failure does not revert the Airdrop call. The bitmap write and the success result remain committed, so the claimant cannot retry after the Airdrop is funded.

An attacker does not need to forge a proof: a valid claimant can submit a valid claim while the configured Airdrop token returns `false` (for example, when the Airdrop is temporarily underfunded). In the batch path, one failed transfer also does not stop later claims from being marked and reported as successful.

Impact Cascade

- A valid claimant permanently loses the recorded claim amount without receiving tokens.
- Batch claims can emit `Claimed` events for entries whose transfers did not settle.
- The bitmap no longer represents paid entitlements, causing irreversible misattribution of the remaining distribution balance.

Validation

- Is the path public and permissionless? Yes. Any caller with a valid Merkle proof can invoke `claim` or `claim_128`.
- Is the token result attacker-controlled or otherwise reachable under supported semantics? A standard ERC20-like transfer may return `false` rather than revert, and the configured token balance can be insufficient at claim time.
- Does state commit if transfer returns `false`? Yes. Neither path asserts the result; both write claim state and emit success.
- Is this dependent on fee-on-transfer or reentrancy? No. The failure is a plain false-return transfer and does not require a token to debit a fee or call back.
- Did existing tests cover the failure result? No. `TestToken.transfer` always asserts balance and returns `true`; the tests therefore only exercise successful transfers and reverting insufficient-balance behavior.

Valid Argument

The contract's public interface promises idempotent claims and returns a processed result, but the bitmap is advanced even when the required token transfer reports failure. This permanently destroys a valid entitlement and is an accounting failure at the distribution boundary.

Invalid Counter-Argument

A triager may argue that deployed Airdrops use a token implementation that reverts on insufficient balance rather than returning `false`, or that deployments are always fully funded. The source interface nevertheless accepts a boolean result, the contract does not enforce the stronger token behavior, and funding can be temporarily insufficient during permissionless claiming. The fix is local and required for the stated ERC20-like interface contract.

Why did tests miss this issue?

The test token's `transfer` reverts on insufficient balance and returns `true` on success. No test token or test case models a `false` return, so the unchecked-result path is absent from coverage.

Recommendation

Require `token.transfer(...)` to return `true` before writing the bitmap or emitting `Claimed`. In the batch path, either perform and verify each transfer before committing its bit or revert the entire transaction on any false result. A safe wrapper should treat both a revert and a false return as a failed claim.

References

- governance/src/airdrop.cairo#L167-L185
- governance/src/airdrop.cairo#L189-L240
- governance/src/interfaces/erc20.cairo#L4-L11
- governance/src/airdrop_test.cairo#L84-L132
- governance/src/airdrop_test.cairo#L587-L624

Client Feedback

Any token used with this Airdrop contract should simply revert if the transfer fails.

4.2.4 EKUB3-48 - Child proxies execute later authorized messages before earlier nonce-ordered messages



Status: Acknowledged

Original severity: Informational

Executive Summary

The L1 `StarknetOwnerProxy` has a sequential `currentNonce`, but that ordering ends when it creates an Arbitrum retryable or an OP Stack cross-domain message. `ArbitrumOwnerProxy` and `OPStackOwnerProxy` do not store or check a child-side nonce. Therefore, accepted messages that were created in nonce order can be redeemed on the child in a different order. This can turn a call that should fail after an earlier authorization-state change into a successful asset movement. For example, governance can queue an ownership transfer away from the `StarknetOwnerProxy` followed by a later value-bearing `execute` call. In the intended order the later call is rejected under the new owner; an unprivileged relayer can redeem the later child message first, while the old owner is still authorized, and drain the proxy. The target and calldata are not substituted; the failure is missing cross-domain execution-order binding.

Details

L1 enforces only its own message-consumption sequence:

```
if (currentNonce != nonce) revert InvalidNonce(currentNonce, nonce);
currentNonce++;
l2MessageBridge.consumeMessageFromL2(l2Owner, getPayload(target, value, nonce, data));
(bool success, bytes memory result) = target.call{value: value}(data);
```

Once the L1 owner call creates child messages, the child proxy has no sequence field:

```
function execute(address target, uint256 value, bytes calldata data) external onlyOwner returns (
    bytes memory) {
    if (target == address(0)) revert InvalidTarget();
    if (address(this).balance < value) revert InsufficientBalance();

    (bool success, bytes memory result) = target.call{value: value}(data);
    if (!success) revert CallFailed(result);
    return result;
}
```

A concrete path using only normal bridge behavior is:

- The L2 Governor authorizes message A, whose L1 execution creates a child message to call `transferOwnership(newOwner)` on the child proxy. It receives child sequence position A.
- The Governor authorizes message B after A, whose L1 execution creates a child message to call `execute(asset, amount, transferData)` from the proxy. It receives child sequence position B.
- The intended child order is A then B. After A, the old `StarknetOwnerProxy` identity is no longer accepted, so B must revert.
- The Arbitrum retryable or OP message for B is independently redeemable. A public relayer redeems B before A. At that time `_checkOwner()` still accepts the old owner alias/messenger sender, so B transfers the proxy-held value. A can then be redeemed and complete the ownership transition.

The same issue affects any order-dependent pair, such as revocation followed by a pending transfer, allowance/state update followed by a dependent action, or upgrade followed by a call intended for the new implementation. The child proxy's arbitrary target call makes the value-bearing ownership-transfer example directly material.

Validation

Five probing questions

- Does `StarknetOwnerProxy.currentNonce` impose order on child redemption? No; it orders only calls to the L1 proxy and the creation of downstream messages.
- Do either child proxies compare an incoming message with a child nonce or sequence? No; `execute` accepts only target, value, and data.
- Can child messages be redeemed independently after L1 creation? Yes, this is the normal retryable/cross-domain message model and requires no forged bridge input.
- Does ownership transfer affect authorization at the child? Yes; `_checkOwner()` derives acceptance from the current owner or its alias/messenger sender.
- Can the later message move real value before the earlier transfer? Yes; `execute` forwards arbitrary value and calldata from the proxy.

Valid argument

The L1 sequence check is not propagated to the child execution layer. A relayer can select the redemption order of two authentic, already-accepted child messages. In the ownership-transfer sequence, the order determines whether the value-bearing call runs under the old or new authorization state. Because the later call is executed before the revocation/transfer, the result is an owner-equivalent asset movement that would not be possible in the nonce order represented by the L1 messages.

Invalid counter-argument

> Each call was authorized by governance, so executing either call is valid; any issue is only relay timing.

This does not explain why the L1 nonce exists or why the messages are emitted in a defined order. The later call is specifically intended to run after the earlier owner transition and would revert in that order. The relayer is not merely delaying a call or causing denial of service: by choosing the opposite order it causes a value transfer to succeed before authorization is revoked.

Determination

Valid. The missing child-side sequence binding changes the state transition and enables an authenticated, public-relayer ordering attack without bridge compromise or payload mutation.

Impact Cascade

- L1 message nonce order no longer establishes execution order on Arbitrum or OP Stack.
- A later call can run while an earlier ownership or revocation message is still pending.
- The proxy's native balance or arbitrary target-controlled assets can be transferred before the earlier authorization change takes effect.
- State-dependent governance batches can finish in a different state than the ordered L1 message stream.

Assumptions and Uncertainties

- Governance has authorized two order-dependent calls and assigned their L1 proxy nonces in the intended order.
- Both downstream messages have been created by the legitimate bridge path and remain redeemable concurrently.
- The child proxy has a value balance or controls a target whose state is affected by the order.
- The report does not assume the relayer can forge, duplicate, or alter a bridge message.

Why did tests miss this issue?

The tests check that an old owner fails and a new owner succeeds when `transferOwnership` is executed immediately. They do not create two authentic pending cross-domain messages and redeem the second before the first. No test asserts that the L1 nonce is enforced again at the child proxy.

Recommendation

Include the L1 nonce or a dedicated child sequence in the downstream message and require the child proxy to consume messages strictly in sequence. For Arbitrum, bind the retryable call to the expected sequence and reject redemption until prior sequence numbers have executed. For OP Stack, add equivalent sequence state to the proxy payload and invalidate/reject out-of-order messages. If strict ordering is not desired, each state-changing action must instead be bound to a current owner epoch and old queued messages must be explicitly revoked.

References

- [governance/l1_proxy/src/StarknetOwnerProxy.sol#L49-L61](#)
- [governance/l1_proxy/src/L1L2OwnerProxy.sol#L16-L26](#)
- [governance/l1_proxy/src/ArbitrumOwnerProxy.sol#L25-L30](#)
- [governance/l1_proxy/src/OPStackOwnerProxy.sol#L15-L17](#)
- [governance/l1_proxy/test/ArbitrumOwnerProxy.t.sol#L106-L138](#)

4.2.5 EKUB3-10 - SDK numeric signed-swap nonces round above 2^{53} and collide on-chain

✓ **Informational**

Status: Fixed by [PR #5](#)

Original severity: Informational

Executive Summary

`encodeSignedSwapMeta` publicly accepts nonce as `bigint | number`, but converts a numeric nonce with `BigInt(nonce)` without requiring a safe integer. JavaScript has already rounded any number above `Number.MAX_SAFE_INTEGER` when this conversion occurs. A caller supplying `9007199254740993` therefore receives metadata encoding nonce `9007199254740992`. The Yul route carries those bytes faithfully, and `SignedExclusiveSwap` consumes the rounded nonce in its extension-global bitmap. Distinct controller quotes built from adjacent large numeric nonces can consequently share one on-chain nonce. An unprivileged relayer can fill one public quote first and make the other revert as already consumed, invalidating a quote that the controller believed had a distinct one-time nonce.

Details

The SDK declares the unsafe union and performs the lossy conversion here:

```
// sdk/src/index.ts:358-363, 382-395
export interface EncodeSignedSwapMetaParameters {
  authorizedLocker?: Address;
  deadline: number;
```

```

    fee?: number;
    nonce: bigint | number;
}

const nonceValue = BigInt(nonce);
if (nonceValue < 0n || nonceValue > (1n << 64n) - 1n) {
    throw new Error("nonce must fit into uint64");
}

```

For the intended nonce 9007199254740993 (0x0020000000000001), a JavaScript number has already become 9007199254740992 (0x0020000000000000). With zero deadline, fee, and locker, the intended and actual 32-byte metadata words differ as follows:

```

intended: 0x0000000000000000000020000000000010000000000000000000000000000000
encoded:  0x0000000000000000000020000000000000000000000000000000000000000000

```

The runtime decodes the latter value, not the caller's intended nonce:

```

// lib/evm-contracts/src/types/signedSwapMeta.sol:41-44
function nonce(SignedSwapMeta meta) pure returns (uint64 value) {
    assembly {
        value := and(shr(128, meta), 0xffffffffffffffff)
    }
}

// lib/evm-contracts/src/extensions/SignedExclusiveSwap.sol:332-351
uint256 word = nonce >> 8;
uint8 bit = uint8(nonce & 0xff);
if (nonceBitmap[word].isSet(bit)) revert NonceAlreadyUsed();
nonceBitmap[word] = next;

```

The bitmap is not keyed by pool or quote. If a controller creates two quotes using numeric nonces 9007199254740992 and 9007199254740993, both SDK calls encode the same nonce 9007199254740992. A relayer can submit either valid route through the Yul signed hop first; the later route reaches `_validateNonceAvailable` with the already-set bit and reverts. This is reachable without a privileged role when the signed quote is usable by the canonical router or has an unrestricted locker.

Impact Cascade

- Distinct signed quotes unexpectedly share one one-time nonce.
- The first public fill consumes the shared extension-global bitmap bit.
- A later quote, potentially for a different pool or user, is rejected as `NonceAlreadyUsed`.
- Quote execution and controller order flow can be selectively invalidated by an unprivileged relayer.

Assumptions and Uncertainties

- The application passes at least one nonce above `Number.MAX_SAFE_INTEGER` as a JavaScript number, which the published type explicitly permits.
- Two quotes use distinct intended numeric nonces that collapse to the same IEEE-754 value; JSON-based nonce transport is a realistic source of this condition.
- The relevant signed quote can be filled by the relayer through its authorized locker, as is the case for a public Yul-router quote or a wildcard locker.
- The issue is avoided when callers use `bigint` for all `uint64` nonces.

How will the bug recipient respond?

> "Callers should use `bigint` for large nonces."

The SDK type explicitly advertises `number`, performs no safe-integer validation, and reports only a `uint64` range check. A supported input type must not silently encode a different nonce and corrupt the runtime's one-time authorization state.

Why did tests miss this issue?

Existing SDK tests use small numeric nonces and one `bigint` nonce, so every tested number is exactly representable. No boundary case exercises `Number.MAX_SAFE_INTEGER` or verifies the encoded nonce against the intended `uint64` value.

Recommendation

Require `bigint` for `nonce`, or reject numeric values unless `Number.isSafeInteger(nonce)` is true before conversion. Prefer the former for `uint64` APIs:

```
nonce: bigint;
```

If numeric compatibility must remain, add an explicit safe-integer check and document that all larger values must be supplied as `bigint`.

References

- [yul-router/sdk/src/index.ts#L358-L395](#)
- [yul-router/lib/evm-contracts/src/types/signedSwapMeta.sol#L41-L44](#)
- [yul-router/lib/evm-contracts/src/extensions/SignedExclusiveSwap.sol#L332-L351](#)
- [yul-router/src/YulRouter.yul#L221-L236](#)
- [sdk/src/index.ts#L358-L363](#)
- [sdk/src/index.ts#L382-L385](#)
- [lib/evm-contracts/src/types/signedSwapMeta.sol#L41-L45](#)
- [lib/evm-contracts/src/extensions/SignedExclusiveSwap.sol#L332-L335](#)

Client Feedback

`bigint` was still supported

4.2.6 EKUB3-46 - Ve33 zero-movement exact-input swaps bypass voter-fee accounting

✓ **Informational**

Status: Acknowledged

Original severity: Informational

Executive Summary

Ve33 requires Core's configured pool fee to be zero and expects all swap fees to be accounted by the extension. However, an exact-input swap whose compact fixed-point price calculation cannot represent any movement enters Core's zero-movement branch. Core consumes the entire specified input and writes it into Core's pool fee-per-liquidity growth. Ve33 then receives a balance update with a zero output delta, so its unspecified-token fee condition does not run: no Ve33 saved pool-fee balance is increased and no voter fee growth is recorded. The router nevertheless settles the input from the caller and returns zero output. The input is therefore diverted into Core fee growth outside Ve33's voter-fee and reward accounting, and `Ve33Positions` has no Core-fee collection path for the affected LP position.

Details

Ve33 explicitly requires zero Core fees:

```
function checkValidPoolKey(PoolKey memory poolKey) private view {
    if (poolKey.config.fee() != 0) revert FeeMustBeZero();
    // ...
}
```

Core's swap loop has a separate exact-input branch when the represented next ratio equals the current ratio:

```
} else {
    // for an exact output swap, the price should always move
    assert(!isExactOut);

    // consume the entire input amount as fees since the price did not move
    assembly ("memory-safe") {
        stepFeesPerLiquidity := div(shl(128, amountRemaining), stepLiquidity)
    }
    amountRemaining = 0;
    sqrtRatioNext = sqrtRatio;
}
```

The loop later stores this input-derived fee growth for the specified input token, while the final balance update still contains the full positive specified input and a zero calculated output.

Ve33's forward path only recognizes a fee when the opposite-token delta is nonzero:

```
(balanceUpdate, stateAfter) = CORE.swap(0, poolKey, params);
// exact input charges the unspecified/output token
if (feeTokenDelta != 0 && exactOut == (feeTokenDelta > 0)) {
    feeAmount = computeFee(amount, swapFee);
    // ... adjust balanceUpdate and save voter fees
}
```

In the zero-movement case `feeTokenDelta` is zero, so `_updatePoolFeeSavedBalances()` and the `poolFeeGrowth` write are skipped. The router's exact-input settlement still calls `payFrom` for the positive specified-token delta. This creates a reachable mismatch between the token debt settled by the caller and the fee bucket used by Ve33 voters. The Core fee growth is not exposed by `Ve33Positions`, whose manager is documented as handling Ve33 reward claims rather than Core swap-fee collection.

A caller can reach this path through the public `swapAllowPartialFill` entrypoint with a positive amount and a normal price limit. The compact ratio has finite precision, so sufficiently small price-impact amounts relative to active liquidity produce `sqrtRatioNextFromAmount == sqrtRatio`; the threshold is determined by liquidity and the current ratio rather than being limited to a single minimal token unit. Standard ERC20s are sufficient.

Validation Questions and Arguments

- Can the zero-movement branch be reached with a nonzero exact-input amount? Valid: Core explicitly enters it when the computed next ratio equals the current ratio and asserts only that the swap is not exact-output. Invalid: a price limit that forces a represented movement avoids this branch, but callers control the amount and limit.
- Does Core account the input anyway? Valid: `stepFeesPerLiquidity` is computed from the entire remaining input, `amountRemaining` becomes zero, and the input-token fee-per-liquidity slot is stored. Invalid: the result is not emitted as a Ve33 fee, but that does not undo Core's fee-growth write.
- Does Ve33 create a corresponding voter-fee balance? Valid: its condition requires a nonzero unspecified-token delta, which is zero when Core returns no calculated output; invalid: if a normal movement occurs, Ve33's ordinary fee path does run.
- Does settlement transfer the input from the caller? Valid: `BaseRouter` settles a positive specified-token delta with `ACCOUNTANT.payFrom`; invalid: the operation can be rejected by a positive minimum-output threshold, but `swapAllowPartialFill` intentionally permits the zero output result.
- Is the result confined to malicious-token behavior or one-unit dust? Valid: only ordinary fixed-point rounding and a standard token are required, and the represented movement threshold scales with active liquidity; invalid: very low-liquidity pools may make the threshold negligible and the affected fee remains pool-local.

Impact Cascade

- A caller pays a nonzero input while the Ve33 swap returns no output.
- Core records the input as Core fee growth despite the Ve33 pool's zero-Core-fee configuration.
- Ve33 voter-fee saved balances and voter fee growth remain unchanged, so the extension's fee/reward accounting does not reconcile with the settled token delta.
- The affected input is not claimable through `Ve33Positions`' voter-fee or LP-reward paths, leaving value unattributed to the intended Ve33 beneficiaries.

Assumptions and Uncertainties

- The pool is a valid Ve33 pool with zero configured Core fee and nonzero active liquidity.
- The exact-input amount is below the compact-ratio movement threshold for the current liquidity and price.
- The caller uses a route that permits partial/zero output, such as `swapAllowPartialFill`.
- The affected Core fee growth is not separately collected through an out-of-band position-management integration.

How will the bug recipient respond?

> "The branch intentionally treats an input that cannot move the represented price as a fee, and users should request a positive minimum output."

That behavior is inconsistent with the Ve33 pool contract's zero-Core-fee model: the input is written to Core fee growth while the extension's only fee accounting path is skipped. A minimum-output check can

prevent one caller from taking the route, but it does not reconcile already accepted zero-output swaps or preserve Ve33 voter accounting when the public partial-fill path is used.

Why did tests miss this issue?

Ve33 fee tests use swap amounts large enough to move the compact price representation and assert the normal opposite-token fee branch. They do not combine a zero-Core-fee Ve33 pool, a representational no-movement exact-input amount, and the partial-fill settlement path.

Recommendation

Do not let a Ve33 pool's zero-movement exact-input branch create Core fee growth. Prefer reverting nonzero Ve33 swaps that cannot produce a represented price movement, or explicitly route the resulting input through a documented Ve33 fee bucket and voter-growth calculation. Add an invariant that every positive specified-token delta in a Ve33 swap either has a corresponding output/extension fee path or is rejected before settlement.

References

- `evm-contracts/src/extensions/Ve33.sol#L359-L366`
- `evm-contracts/src/extensions/Ve33.sol#L543-L585`
- `evm-contracts/src/Core.sol#L702-L738`
- `evm-contracts/src/Core.sol#L832-L850`
- `evm-contracts/src/base/BaseRouter.sol#L330-L344`
- `src/extensions/Ve33.sol#L357-L360`
- `src/Core.sol#L702-L712`
- `src/extensions/Ve33.sol#L546-L551`

Client Feedback

This case is extremely rare and does not happen in practice, and since it requires the amount to be very small in practice the collected fees have no impact except for wasted gas